

Lecture 6: Relational Algebra to SQL

A Practical Guide

Amit Kumar Dhar

August 8, 2025

Introduction

Our Running Example: Student Database

Students

sid	sname	discip
101	Isha	CSE
102	Parth	DSAI
103	Juhi	CSE
104	Suresh	MA

Enrolled

sid	cid	grade
101	CSL303	A
102	DSL201	B
103	CSL303	B
101	CSL211	A

Fundamental Operators in SQL

Selection (σ) $\rightarrow$ WHERE

Concept: The 'WHERE' clause in SQL is the direct equivalent of the selection operator.

SQL:

```
SELECT *  
FROM Students  
WHERE discip = 'CSE';
```

Relational Algebra:

$\sigma_{\text{discip}='CSE'}(\text{Students})$

Result:

sid	sname	discip
101	Isha	CSE
103	Juhi	CSE

Projection (π) $\rightarrow$ SELECT

Concept: The 'SELECT' clause in SQL corresponds to projection.

Key Difference: SQL's 'SELECT' **does not** eliminate duplicates by default. Relational algebra's π **does**. Use 'SELECT DISTINCT'.

SQL:

Relational Algebra:

$\pi_{\text{discip}}(\text{Students})$

-- Keeps duplicates:

```
SELECT discip  
FROM Students;
```

-- Eliminates duplicates:

```
SELECT DISTINCT discip  
FROM Students;
```

Projection Example 2

Concept: Projecting multiple columns is straightforward.

SQL:

```
SELECT sname, discip  
FROM Students;
```

Relational Algebra:

$\pi_{\text{sname, discip}}(\text{Students})$

Union ($\cup$) $\rightarrow$ UNION

Concept: The 'UNION' operator in SQL combines the results of two queries.

Key Difference: 'UNION' automatically removes duplicates. To keep all rows, you must use 'UNION ALL'.

SQL:

Relational Algebra:

$\pi_{\text{sid}}(\sigma_{\text{discip}='CSE'}) \cup \pi_{\text{sid}}(\sigma_{\text{grade}='A'})$

```
(SELECT sid FROM Students
 WHERE discip = 'CSE')
UNION
(SELECT sid FROM Enrolled
 WHERE grade = 'A');
```


Intersection ($\cap$) $\rightarrow$ INTERSECT

Concept: The 'INTERSECT' operator finds rows common to two query results.

Key Difference: Like 'UNION', 'INTERSECT' also removes duplicates.

SQL:

Relational Algebra:

$\pi_{\text{sid}}(\sigma_{\text{discip}='CSE'}) \cap \pi_{\text{sid}}(\sigma_{\text{grade}='A'})$

```
(SELECT sid FROM Students  
WHERE discip = 'CSE')  
INTERSECT  
(SELECT sid FROM Enrolled  
WHERE grade = 'A');
```

Cartesian Product ($\times$) $\rightarrow$ CROSS JOIN

Concept: 'CROSS JOIN' produces every combination of rows from two tables.

Warning: This operation can produce huge result sets and should be used with caution. It is rarely used in practice compared to other joins.

SQL:

Relational Algebra:

$\sigma_{\text{discip}='CSE'} \times \sigma_{\text{cid}='CSL211'}$

```
SELECT * FROM  
(SELECT * FROM Students  
  WHERE discip='CSE') AS S1  
CROSS JOIN  
(SELECT * FROM Enrolled  
  WHERE cid='CSL211') AS E1;
```

The Join Family in SQL

Inner Join ($\bowtie$) $\rightarrow$ INNER JOIN

Concept: The modern 'INNER JOIN ... ON' syntax is the standard way to perform an equi-join.

SQL:

Relational Algebra:

Students $\bowtie_{\text{sid}}$ Enrolled

```
SELECT *  
FROM Students  
INNER JOIN Enrolled  
ON Students.sid = Enrolled.sid;
```

Inner Join using WHERE

Concept: An older, implicit syntax for joins uses a comma-separated list of tables in the 'FROM' clause and the join condition in the 'WHERE' clause.

Warning: This syntax is discouraged. It's easy to forget the join condition, accidentally producing a Cartesian product. The modern 'JOIN' syntax is clearer and safer.

SQL (Old Syntax):

Relational Algebra:

Students $\bowtie_{\text{sid}}$ Enrolled

```
SELECT *  
FROM Students, Enrolled  
WHERE Students.sid = Enrolled.sid
```

Natural Join → NATURAL JOIN

Concept: SQL provides a 'NATURAL JOIN' keyword that automatically joins on all columns with the same name.

Warning: 'NATURAL JOIN' can be dangerous! It may join on unexpected columns if the schema changes. It is often better to be explicit.

SQL:

```
SELECT *  
FROM Students  
NATURAL JOIN Enrolled;
```

Relational Algebra:

Students $\bowtie$ Enrolled

Outer Joins in SQL

Outer joins are critical for finding data that *doesn't* have a match in another table. The SQL syntax is very expressive.

- **Left Outer Join:** Keeps all rows from the **left** table.

Students $\bowtie$ Enrolled $\rightarrow$ LEFT OUTER JOIN

- **Right Outer Join:** Keeps all rows from the **right** table.

Students $\bowtie$ Enrolled $\rightarrow$ RIGHT OUTER JOIN

- **Full Outer Join:** Keeps all rows from **both** tables.

Students $\bowtie$ Enrolled $\rightarrow$ FULL OUTER JOIN

The 'ON' clause specifies the join condition, just like with 'INNER JOIN'.

Left Outer Join Example

Query: List all students and the courses they are enrolled in, including students not enrolled in any course.

SQL:

```
SELECT s.sname, e.cid  
FROM Students AS s  
LEFT OUTER JOIN Enrolled AS e  
ON s.sid = e.sid;
```

Relational Algebra:

Students $\bowtie$ Enrolled

Result:

sname	cid
Isha	CSL303
Isha	CSL211
Parth	DSL201
Juhi	CSL303
Suresh	NULL

Right Outer Join Example

Query: List all enrollments and the students they belong to, including enrollments that don't match any student.

SQL:

Relational Algebra:

Students $\bowtie$ Enrolled

```
SELECT s.sname, e.cid  
FROM Students AS s  
RIGHT OUTER JOIN Enrolled AS e  
ON s.sid = e.sid;
```

Full Outer Join Example

Query: List all students and all enrollments, matching them where possible.

SQL:

```
SELECT s.sname, e.cid  
FROM Students AS s  
FULL OUTER JOIN Enrolled AS e  
ON s.sid = e.sid;
```

Relational Algebra:

Students $\bowtie$ Enrolled

Advanced Operators in SQL

Grouping (γ) $\rightarrow$ GROUP BY

Concept: The 'GROUP BY' clause corresponds to the grouping attributes in the γ operator. Aggregate functions are used in the 'SELECT' list.

SQL:

Relational Algebra:

$\gamma_{\text{discip}, \text{COUNT}(\text{sid})}(\text{Students})$

```
SELECT discip, COUNT(sid)
FROM Students
GROUP BY discip;
```

Filtering Groups: To filter groups (like σ on the result of a γ), SQL uses the 'HAVING' clause.

$\sigma_{\text{num}>1}(\gamma_{\dots}) \rightarrow \text{HAVING COUNT(sid) > 1}$

Sorting (τ) $\rightarrow$ ORDER BY

Concept: The 'ORDER BY' clause is used to sort the final result of a query.

SQL:

```
SELECT *  
FROM Students  
ORDER BY discip, sname;
```

Relational Algebra:

$\tau_{\text{discip, sname}}(\text{Students})$

Complex Queries: RA vs SQL

Query 1: Popular Courses

Query: Find the 'cid' of courses with more than 1 student.

Relational Algebra:

$$\pi_{cid}(\sigma_{num_students > 1}(\gamma_{cid, COUNT(sid) AS num_students}(Enrolled)))$$

SQL:

```
SELECT cid
FROM Enrolled
GROUP BY cid
HAVING COUNT(sid) > 1;
```

Query 2: Top CSE Students

Query: Find the names of 'CSE' students who received an 'A'.

Relational Algebra:

$$\pi_{\text{name}}((\sigma_{\text{discip}='CSE'}(\text{Students})) \bowtie (\sigma_{\text{grade}='A'}(\text{Enrolled})))$$

SQL:

```
SELECT S.sname
FROM Students AS S
JOIN Enrolled AS E ON S.sid = E.sid
WHERE S.discip = 'CSE' AND E.grade = 'A';
```


Thank You!
Questions?